

计算机体系结构

L07 向量处理器

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ④

多处理器 (线程级并行) ⑤

加速器 (数据并行)

向量处理器

体系架构

性能优化

SIMD扩展指令集

每周期处理多个数据元素

变长向量

处理条件语句

Memory Banks

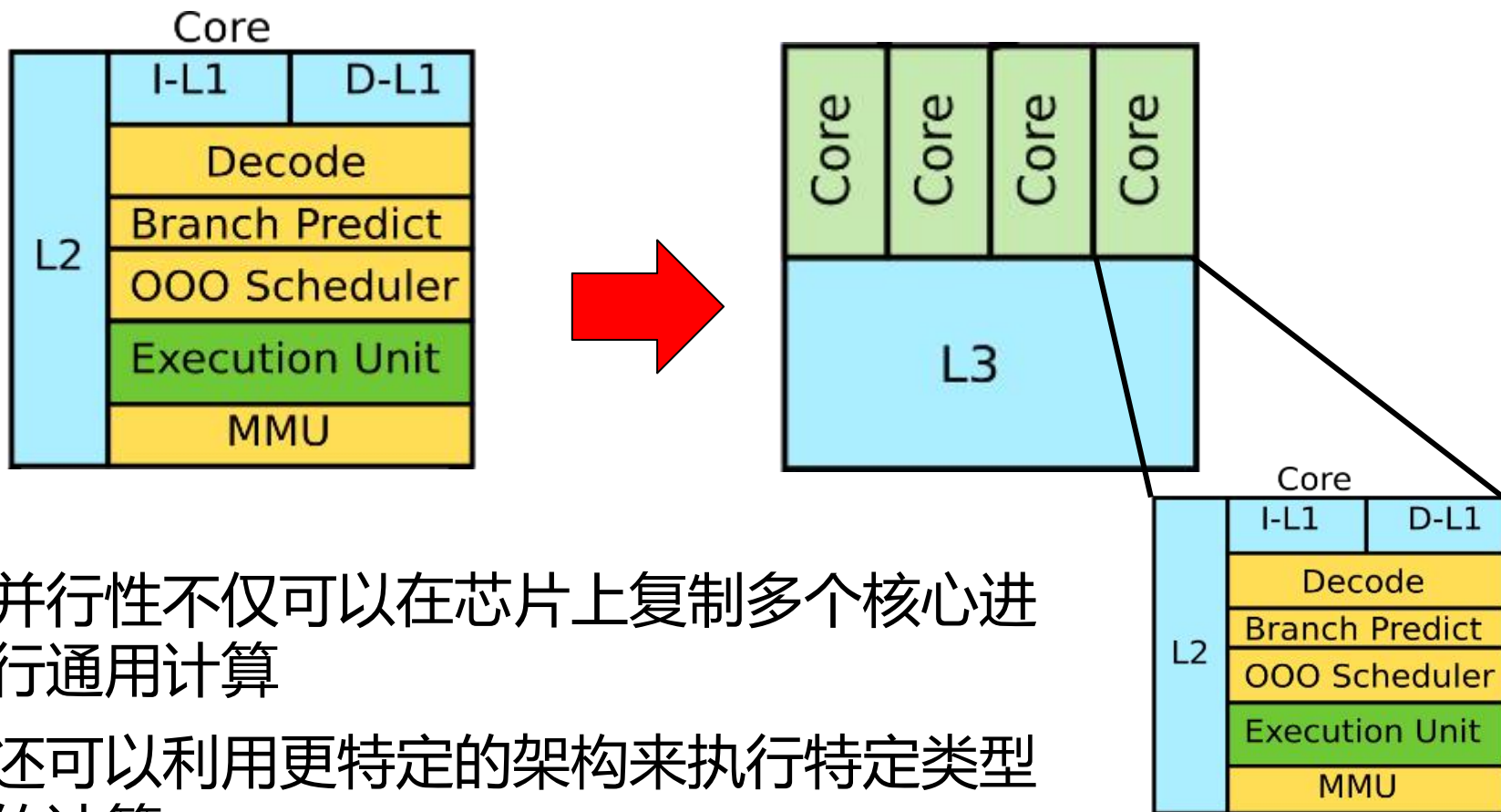
步长(Stride)

分散/聚集操作

GPU ⑤

AI 加速器 ④

并行性不仅仅是从单核到多核



- 并行性不仅可以在芯片上复制多个核心进行通用计算
- 还可以利用更特定的架构来执行特定类型的计算

并行性类型

□ Instruction level parallelism(指令级并行)

- 同一流中不同的指令并行执行
- Pipelining, OOO execution, speculative execution, VLIW

□ Task level parallelism(任务级并行)

- 不同程序并行执行
- 同一程序不同的线程: Java (scheduling, GC, etc...), 显式编程的多线程程序 (pthreads, MPI, ...)
- 多线程硬件/多核

□ Data level parallelism (数据并行)

- 并行处理不同的数据片段, e.g. 图像和视频处理
- 向量计算, 阵列计算, 脉冲阵列

Flynn' s 计算机分类

□ SISD:

- 单指令操作单数据

□ SIMD:

- 单指令流操作多个数据元素, e.g. 阵列或向量处理器 → data parallel

□ MISD:

- 多条指令操作单数据
- 最接近的形式：脉冲阵列处理器，流处理器

□ MIMD:

- 多条指令操作多个数据, e.g. 多核, 多线程

数据并行(Data Level Parallelism)

□ 并行性来自在不同数据片段上做相同的操作

□ Examples:

- 两个矩阵相加
- Sobel算子进行边缘检测

What kind of
FU to use?

0,0	0,1	0,2	0,3	0,4	0,5
1,0	1,1	1,2	1,3		
2,0	2,1	2,2	2,3		
3,0	3,1	3,2	3,3		
4,0					
5,0					

+

-1	0	+1
-2	0	+2
-1	0	+1

x filter

=

0,0	0,1	0,2	0,3	0,4	0,5
1,0	1,1	1,2	1,3		
2,0	2,1	2,2	2,3		
3,0	3,1	3,2	3,3		
4,0					
5,0					

Original image



Edge detected image



利用数据并行性的体系结构

□ 假设晶体管总预算有限

□ 两个稠密浮点矩阵相加

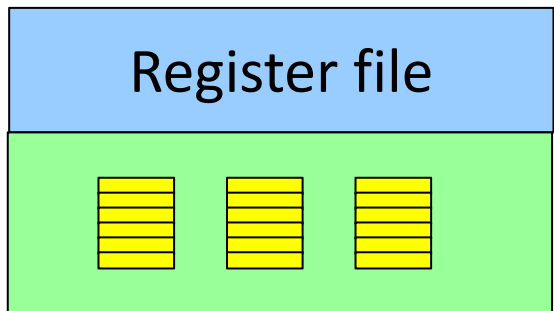
- 每个数据元素进行复杂运算(FP addition)
- **策略:** 计算流水线化, e.g. 6-stage FP adder
 - 与指令流水线不同,不需要解决分支,相关性, 中断

□ Sobel算子进行边缘检测

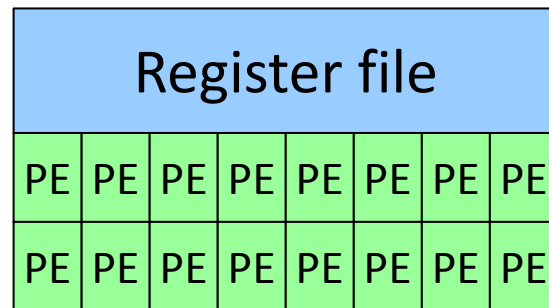
- 每个数据元素进行简单运算 (integer multi)
- **策略:** 大量简单功能单元,每个功能单元处理一个数据,并行工作

利用数据并行性的体系结构

- 流水线式功能单元
(*vector processor*)



- 很多简单功能单元
(*array processor*)



这些功能单元（一个操作多个数据的流水线或者m个处理单元操作m个数据）可以通过单条指令激活
→ SIMD

* 一个PE (processing element)
可能仅是一个简单整数ALU

SIMD 架构类型

- ❑ **Vector architectures(向量处理器)**
 - 一条指令引起多个数据操作的流水线执行
- ❑ **Graphics processing units (GPUs) and array processors (阵列处理器)**
 - 一条指令引起多个数据单元同时执行
- ❑ **Multimedia SIMD instruction set extensions**
 - ISA扩展，用于多媒体应用的并行数据操作

SIMD 架构特点

□ SIMD利用显著的数据级并行性进行特定类型的计算, e.g.

- 面向矩阵的科学计算
- 面向多媒体的图形和视频处理
- 需要特殊的架构设计: vector or array

□ SIMD比MIMD能效更高，计算密度也更高

- 每个数据操作取一条指令
- 可以是一个运算单元,也可以是处理器

□ SIMD允许程序员继续按顺序编程一样思考

向量处理器 **(Vector Processors)**

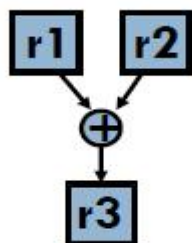
Vector Processors (向量处理器)

□ 很多程序使用vectors

```
for (i=0; i<1000; i++) // DAXPY  
    Y[i] = a * X[i] + Y[i];
```

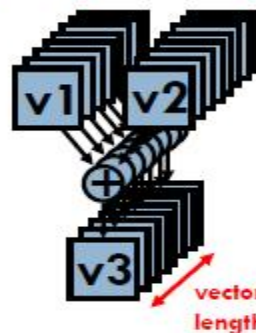
□ 向量处理器指令操作的是向量，而不是标量

SCALAR
(1 operation)



add r3, r1, r2

VECTOR
(N operations)



vadd.vv v3, v1, v2

Vector Processors (向量处理器)

□ 一个向量指令在连续的周期内对每个元素执行操作

- 向量功能单元是流水线的
- 每个流水阶段处理不同数据元素

□ 向量处理器的基本要求

- Load/store 向量 → **vector registers** (contain vectors)
- 操作不同长度的向量 → **vector length register** (VLR)
- 一个向量的元素可能在内存中相互分开存储 → **vector stride register** (VSTR)

向量处理器构成

□ A scalar processor (e.g. a MIPS processor)

- 标量寄存器文件 (32 registers)
- 标量功能单元 (arithmetic, load/store, ...)

□ 向量寄存器文件 (a 2D register array)

- 每个寄存器都是一个向量，例如，32个寄存器，每个寄存器有64个64位元素
- $MVL = \text{最大向量长度} = \text{每个寄存器最大元素个数}$

□ 一组向量功能单元

- Integer, FP, load/store, ...

□ 有时向量和标量单元会结合在一起（共享ALU）

Example Architecture: VMIPS

□ Vector registers

- 每个寄存器包含一个64个64 bits元素的向量
- 寄存器文件有16个读端口和8个写端口

□ Vector functional units

- 完全流水线化
- 检测数据和控制危害

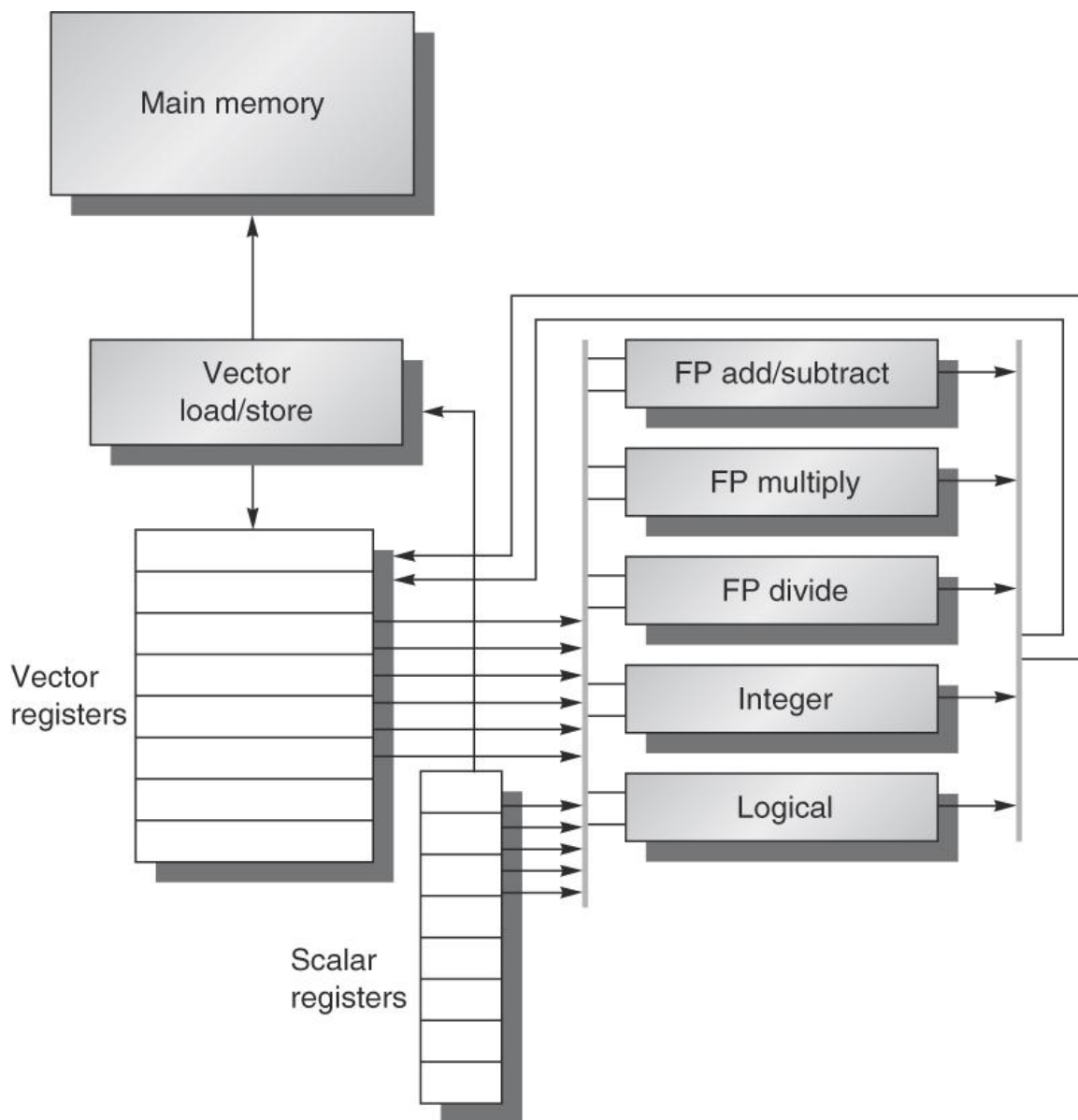
□ Vector load-store unit

- 完全流水线化
- 每个周期处理一个word

□ Scalar registers

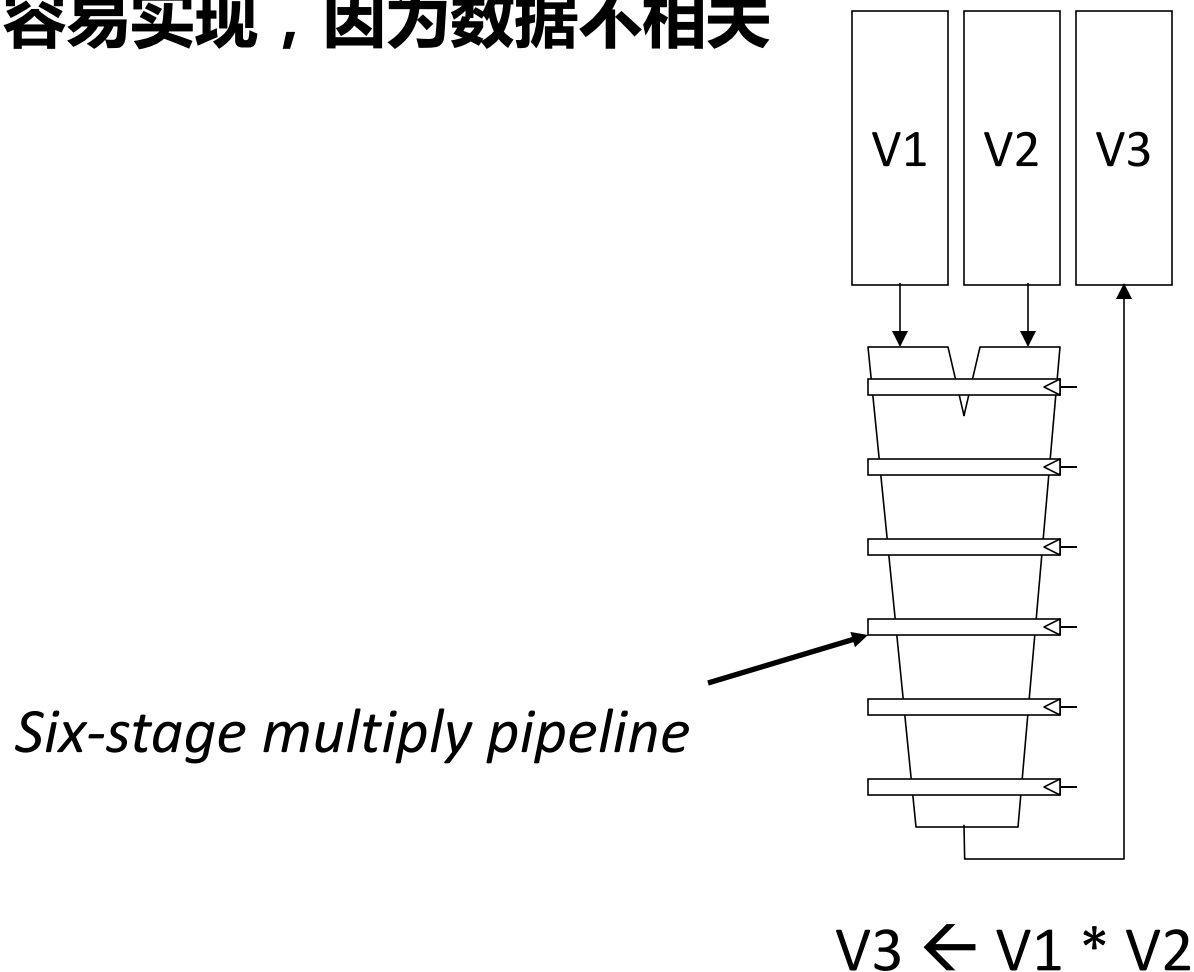
- 32 个通用寄存器
- 32 个浮点寄存器

VMIPS 基本结构



向量功能单元

- 采用深度流水线
- 流水线控制容易实现，因为数据不相关



VMIPS 指令

- ADDVV.D: 向量+向量
- ADDVS.D: 向量+标量
- LV/SV: 内存读向量/写向量

Example: DAXPY ($Y = a * X + Y$)

```
L.D    F0,a        ; load scalar a
LV      V1,Rx       ; load vector X
MULVS.D V2,V1,F0    ; vector-scalar multiply
LV      V3,Ry       ; load vector Y
ADDVV.D V4,V2,V3    ; add
SV      Ry,V4       ; store the result
```

- 需要6条指令 vs. 标量处理器需要600条指令

向量处理器的优点

□ 向量内数据无相关性

- 流水线, 并行性好
- 流水线很深

□ 每条指令做很多工作

- 降低取指令带宽

□ 高度规则的内存访问模式

- 交错多个bank以获得更高的存储器带宽
- 适合预取

□ 无需显式编写循环

- 指令序列中分支较少
- 容易编程

向量处理器的缺点

□ 仅适合数据并行性好的操作

- 如果并行性差则效率非常低
- 遍历链表？

□ 内存带宽容易称为瓶颈

- 计算/访存平衡没有掌握好的话
- 数据到bank的映射没有做好的话

向量执行时间

□ 执行时间取决于三个因素:

- 向量长度
- 结构冲突
- 数据相关性

□ VMIPS功能单元每个时钟周期完成一个元素

- 执行时间近似等于向量长度

□ 如何估算一组向量指令的执行时间?

- 直观地: 向量指令的数量 $\times$ 向量长度

链式执行

□ 在直观模型中，一个向量指令必须在下一条指令可以开始执行之前完成其执行

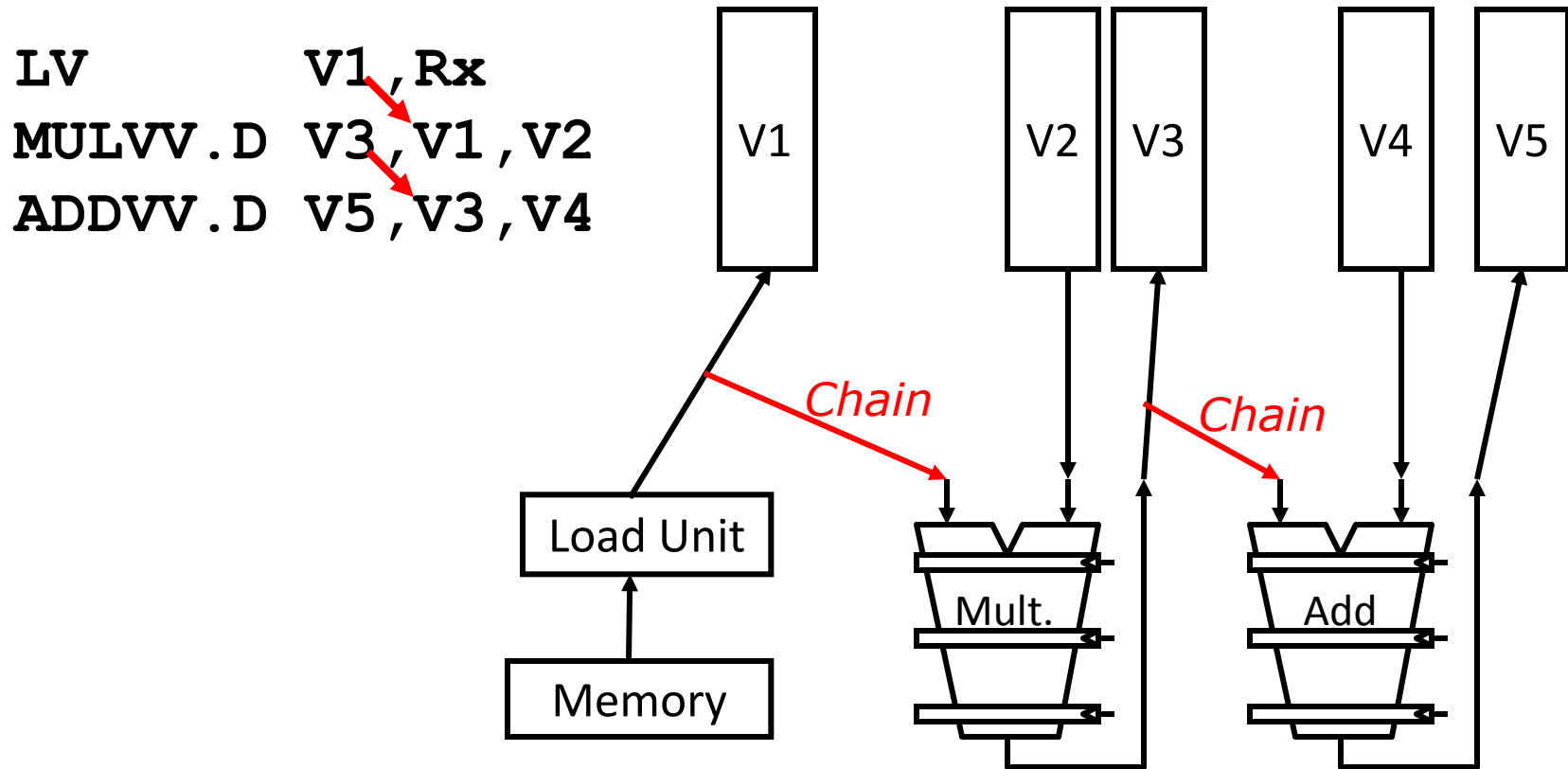
- 一个向量功能单元的输出不能用作另外一个单元的输入 (i.e., no vector data forwarding)

□ 链式执行

- 允许一个向量操作在其向量源操作数从另一个向量功能单元变得可用时立即开始
- 具有RAW依赖冲突的向量指令序列可以进行链接

链式执行

□ 从一个功能单元到另外一个的数据传递

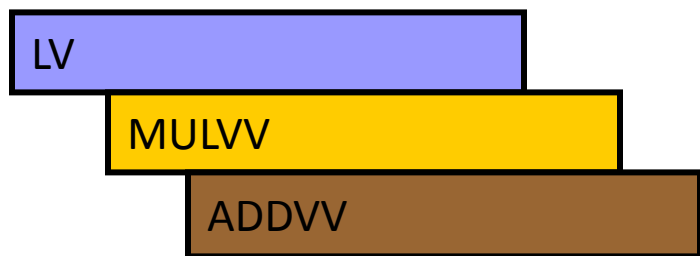


链式执行的优点

- **不使用链式操作的情况下**，必须等到结果的最后一个元素写入向量寄存器后，才能开始执行依赖的指令



- **采用链式操作**，可以在第一次结果出现时就开始执行依赖指令

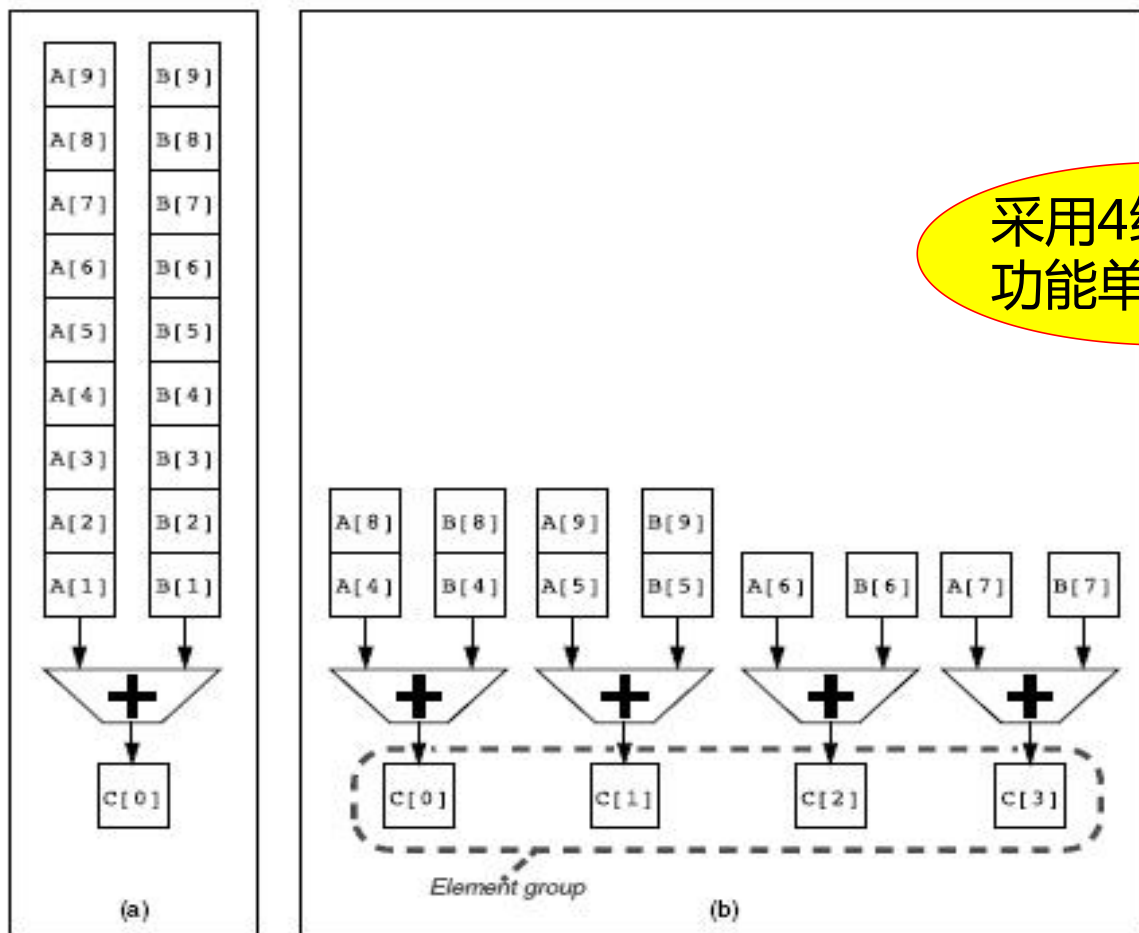


性能优化

- 每周期处理多个数据 (multiple lanes)
- Non-64 wide vectors (register length register)
- IF statements in vector code (vector mask)
- Memory system optimizations to support vector processors (multiple memory banks)
- Multiple dimensional matrices (stride)
- Sparse matrices (gather/scatter)
- Programming a vector computer

1. 每周期处理多个数据元素

□ 采用阵列式处理器架构或者混合式



2. 变长向量

□ 如果向量长度不固定,运行时再确定呢?

- 使用一个向量长度寄存器 (VLR) 控制向量操作的长度
- VLR的值不能超过向量寄存器的长度

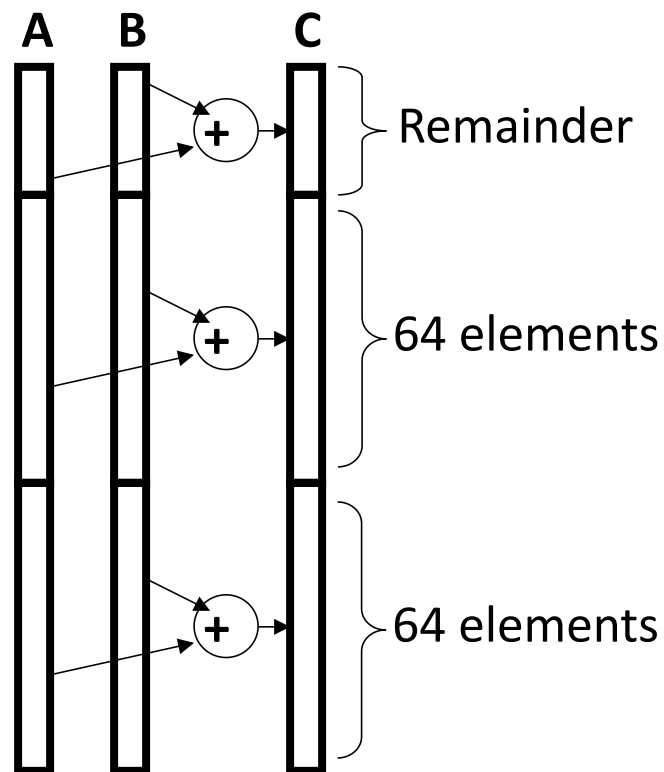
□ 如果向量长度大于向量寄存器的长度呢?

- 需要将向量拆分,多次完成
→ 向量分块

向量分块(Vector Stripmining)

- **Problem:** 向量寄存器长度有限
- **Solution:** 将循环分解为适合寄存器的片段

```
for (i=0; i<N; i++)  
    C[i] = A[i] + B[i];
```



3. 处理条件语句

□ 如果处理条件语句？

```
for (i=0; i<64; i++)  
    if (X[i] != 0)  
        X[i] = X[i] - Y[i];
```

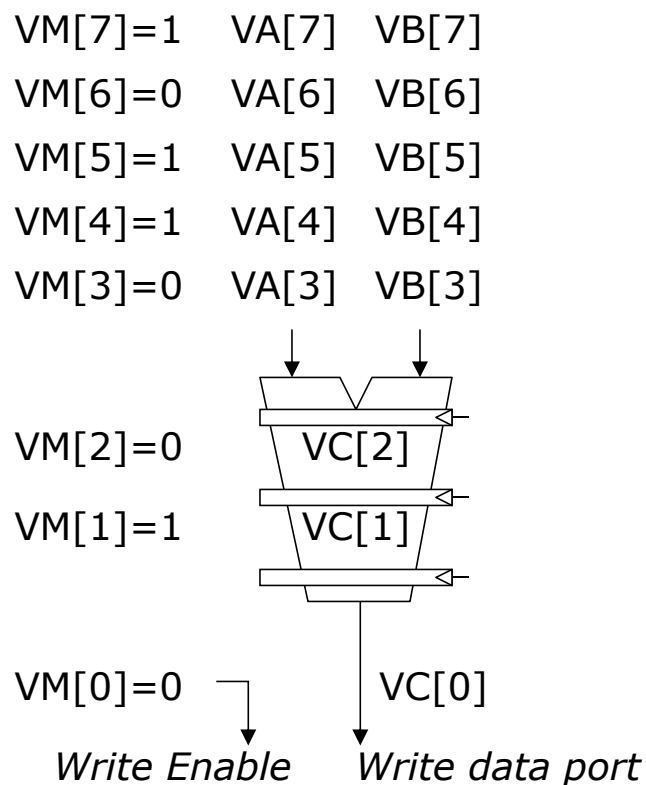
□ **Idea:** 使用向量掩码寄存器(VMR) *标记那些特殊元素.*

```
LV      V1, Rx      ;load vector X into V1  
LV      V2, Ry      ;load vector Y into V2  
L.D      F0, #0      ;load FP zero into F0  
SNEVS.D V1, F0      ;sets VM[i] to 1 if V1[i]!=F0  
SUBVV.D V1, V1, V2   ;subtract under vector mask  
SV      Rx, V1      ;store the result in X
```

带有掩码的向量指令

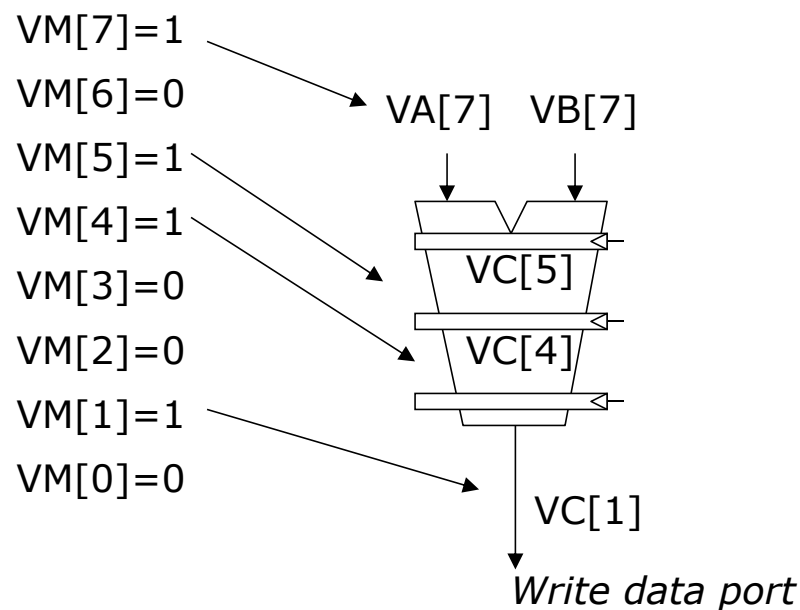
简单实现

- 执行全部指令(包括无效的元素), 最后写回的时候去掉无效元素



更高效的实现

- 根据掩码直接调过无效元素不执行



4. Memory Banks

□ 内存系统必须重新设计以满足高带宽需求

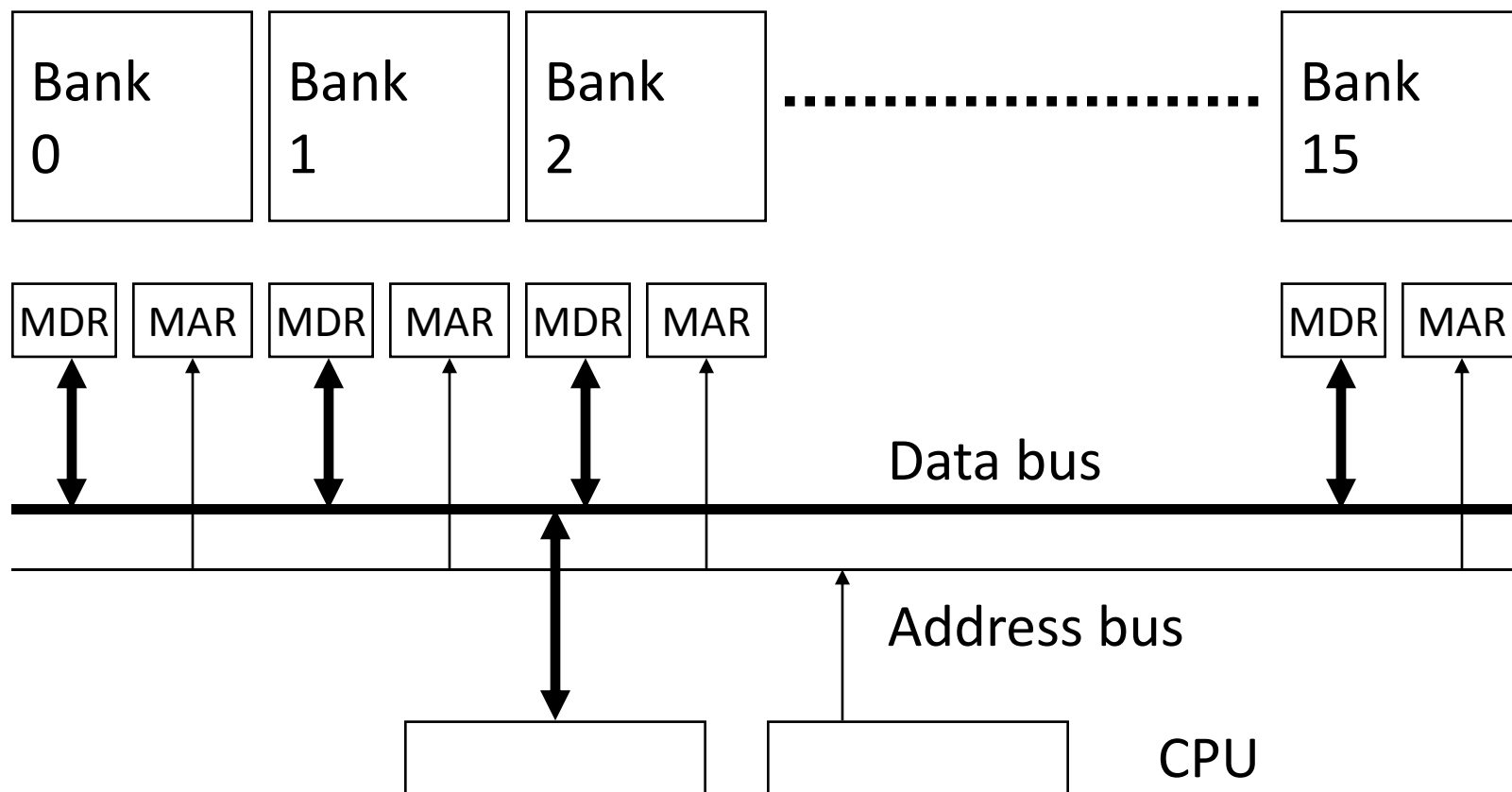
□ 在多个内存bank之间分散访问

- 独立控制bank地址
- Load/store非连续数据
- 支持多个向量处理器共享内存

Memory Banking

□ 每周期可以开始1个band访问

- 可以支撑16个并行访问,如果在不同的banks



5. 步长(Stride)

□ 考虑矩阵乘法:

```
for (i=0; i<100; i++)  
  for (j=0; j<100; j++) {  
    A[i][j] = 0.0;  
    for (k=0; k<100; k++)  
      A[i][j]=A[i][j]+B[i][k]*C[k][j]; }
```

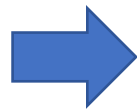
□ 必须对矩阵B的行与矩阵C的列进行向量化乘法

- 采用非单位步长, 使用向量步长寄存器(VSTR)

1	2	3
4	5	6
7	8	9

X

a	b	c
d	e	f
g	h	i



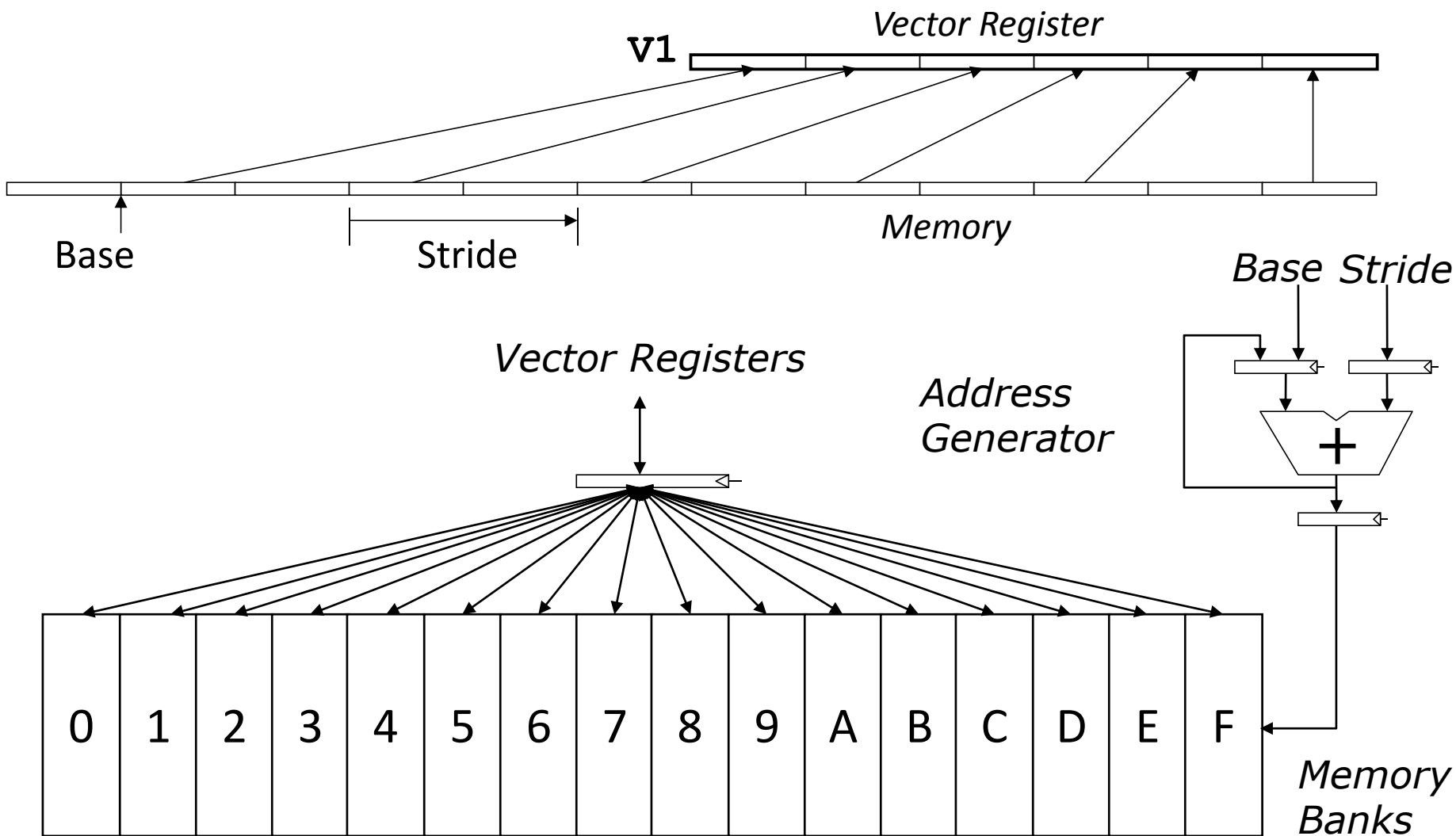
1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

a	b	c	d	e	f	g	h	i
---	---	---	---	---	---	---	---	---



stride

基于步长的内存访问



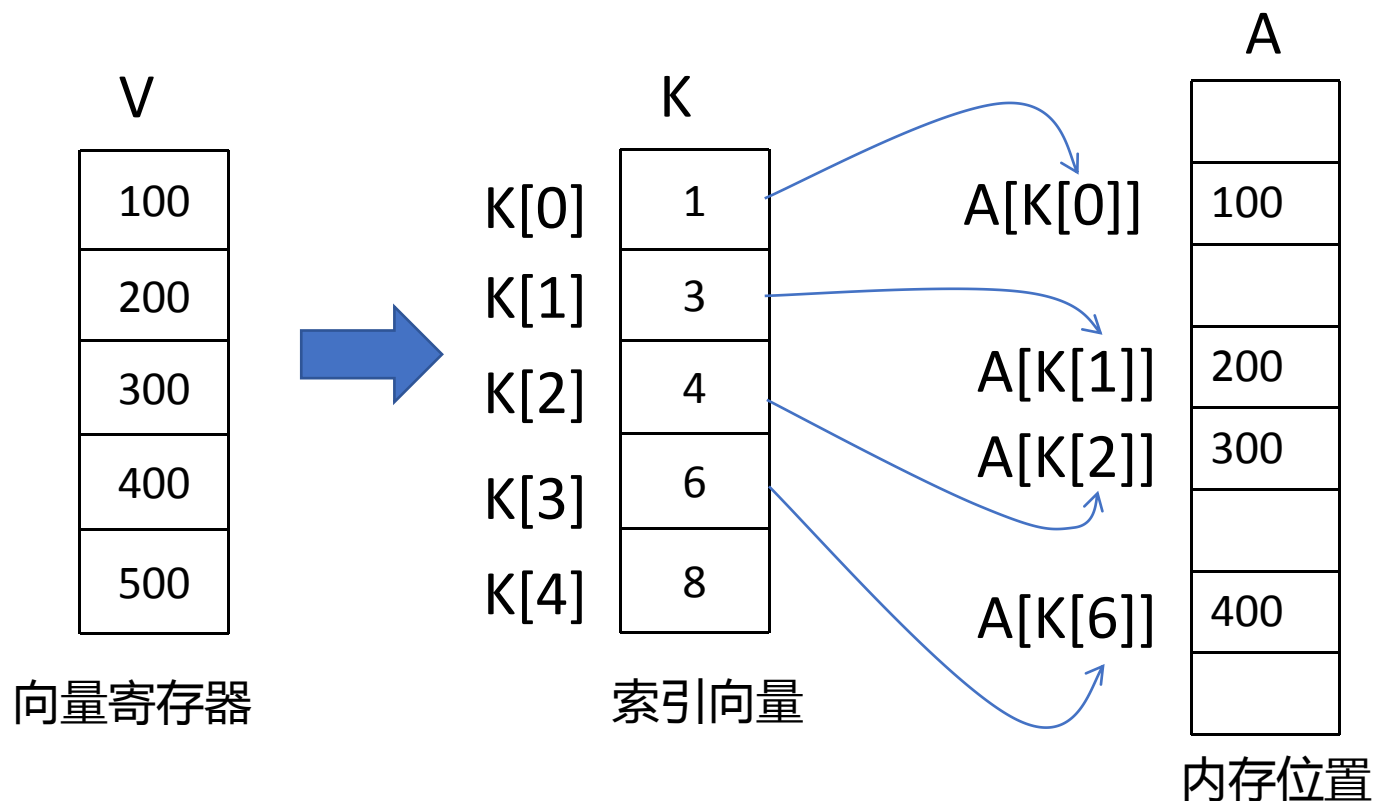
6. 分散/聚集操作

- **分散**：将向量寄存器中的数据存储在内存的非连续地址中，这些地址由一个地址向量（或索引向量）指定。

6. 分散/聚集操作

□ 分散：将向量寄存器中的数据存储在内存的非连续地址中

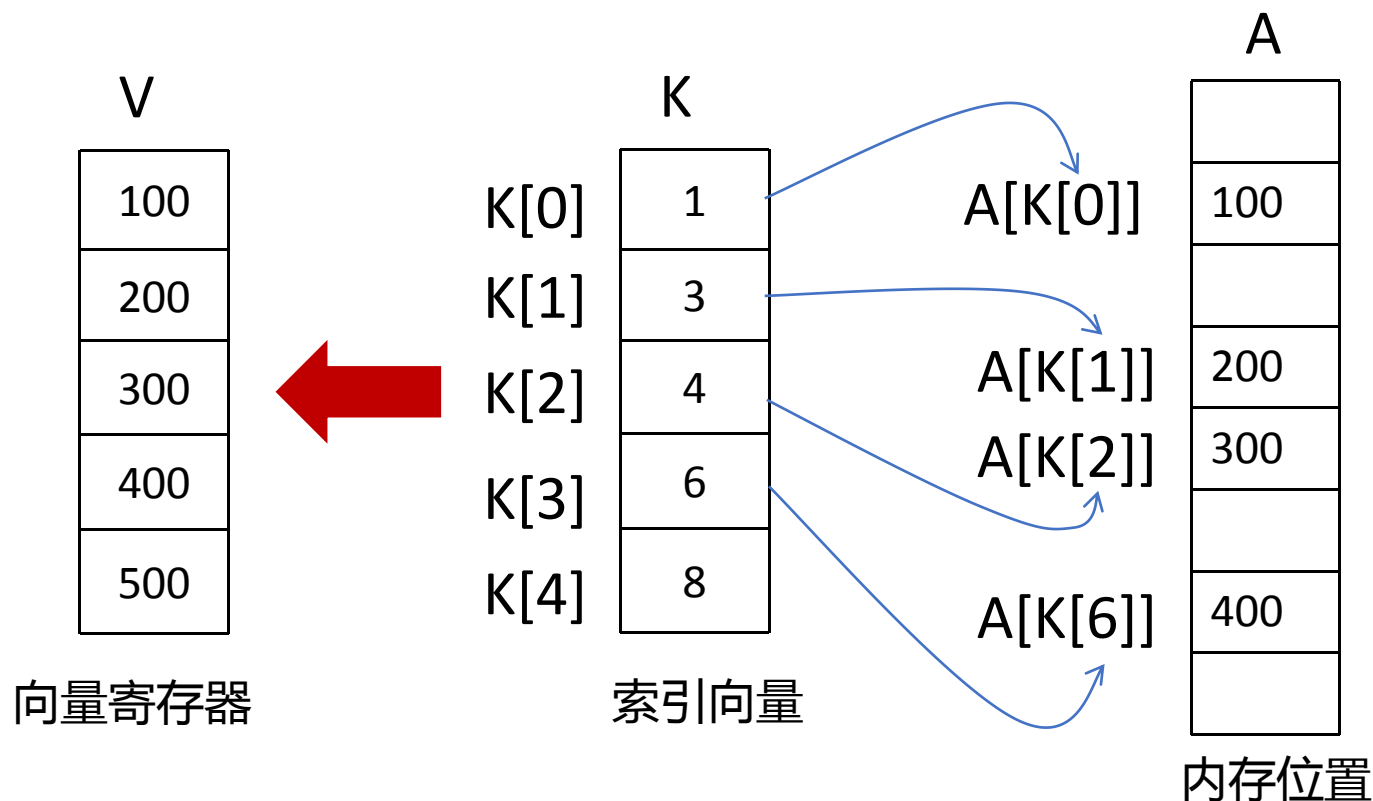
```
for (i=0; i<n; i++)  
    A[K[i]] = A[K[i]] + C[M[i]];
```



6. 分散/聚集操作

□ 聚集：从内存的非连续地址中加载数据到向量寄存器

```
for (i=0; i<n; i++)  
    A[K[i]] = A[K[i]] + C[M[i]];
```



6. 分散/聚集操作

□ 聚集：从内存的非连续地址中加载数据到向量寄存器

```
for (i=0; i<n; i++)  
    A[K[i]] = A[K[i]] + C[M[i]];
```

□ 汇编代码示例

```
LV      Vk, Rk          ;load K[] into Vk (index reg.)  
LVI     Va, (Ra+Vk)      ;load indirect A[K[]]  
LV      Vm, Rm          ;load M[] into Vm (index reg.)  
LVI     Vc, (Rc+Vm)      ;load indirect C[M[]]  
ADDVV.D Va, Va, Vc       ;add them  
SVI     (Ra+Vk), Va      ;store A[K[]]
```

SIMD 扩展指令

(SIMD Instruction Set Extensions)

SIMD 指令集扩展

□ 多媒体程序通常处理比word更短的数据

- e.g. 图形学用8 bits表示RGB

□ 一条指令可以同时操作多个数据元素，操作码(opcode) 确定数据类型:

- 8个8-bit字节, 4个 16-bit字, 2个32-bit 双字, 1个64-bit 4字
- 操作数必须是连续的，并且位于对齐的内存位置

SIMD 指令集扩展

□ Intel MMX (1996)

- 8个8-bit整数操作或者4个16-bit整数操作

□ Streaming SIMD Extensions (SSE) (1999)

- 8个16-bit整数操作
- 4个32-bit 整数/浮点数操作或2个64-bit 整数/浮点数操作

□ Advanced Vector Extensions (AVX) (2010)

- 4个64-bit 整数/浮点数操作 (for 256-bit AVX)
- Example AVX instructions:
 - VADDPD: 4个double数打包到一块相加
 - VMOVAPD: 将4个double数移动到另外一个位置
 - VBROADCASTSD: 将一个double数广播到4个位置(256-bit寄存器指定)

SIMD 扩展指令集的缺点

□ 相比向量指令的缺点:

- 固定个数的数据编码成一个操作码
→ 需要添加大量指令
- 没有复杂的寻址模式 (strided, scatter/gather) → 向量化的机会更少
- 没有掩码寄存器
→ 编程更难

□ 为何SIMD扩展指令集更流行?

- 扩展标准算术单元的成本很低
- 需要较少内存带宽
- 能够轻松处理虚拟内存和缓存